



UNIVERSIDAD CATÓLICA
DE SANTIAGO DE GUAYAQUIL

**FACULTAD DE ECONOMÍA Y EMPRESA CARRERA DE
ADMINISTRACIÓN DE EMPRESAS
CICLO VII**

BIG DATA

TITULO:

COMPARACIÓN DE MODELOS Y OPTIMIZACIÓN DE PIPELINE EN EL
DATASET TITANIC

INTEGRANTES:

VALENTINA VERÓNICA SUÁREZ DE LA TORRE

MARÍA DANIELA BORBOR CABRERA

ANAI BETZABETH BENAVIDES VEGA

RICARDO ALBERTO ZARIQUIEY SALINAS

CHRISTIAN EFREN CEDEÑO SOTOMAYOR

DOCENTE:

ING. XAVIER JÁCOME PIÑEIRO

Guayaquil, Ecuador

Integrante	Tareas realizadas
Valentina	Carga de datos, EDA y feature engineering (Fare_per_person , Deck)
Daniela	Construcción del Pipeline + ColumnTransformer y modelos base
Anaí	Validación cruzada, búsqueda de hiperparámetros y análisis de overfitting
Ricardo y Christian	Evaluación comparativa, gráficos (ROC, matriz de confusión, importancias) e informe

1. Introducción

El objetivo de este trabajo es predecir si un pasajero del Titanic sobrevivió al naufragio a partir de sus características (clase, sexo, edad, tarifa, familia a bordo y puerto de embarque). Es un problema de clasificación binaria con clases desbalanceadas: en los datos de entrenamiento, alrededor del 62 % no sobrevivió frente a un 38 % que sí, un detalle que condiciona cómo medimos el desempeño.

En clases construimos un primer clasificador con imputación básica, codificación de categóricas y un solo modelo, sin comprobar si era la mejor alternativa. Este trabajo va más allá en cuatro aspectos. Primero, encapsulamos todo el preprocesamiento (ingeniería de variables, imputación, escalado y one-hot encoding) dentro de un Pipeline de scikit-learn, de modo que cada estadística se calcula únicamente con datos de entrenamiento en cada partición de la validación cruzada, eliminando el data leakage. Segundo, comparamos cuatro algoritmos bajo las mismas condiciones: Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting. Tercero, optimizamos hiperparámetros de dos de ellos con GridSearchCV y RandomizedSearchCV (**cv=5**). Cuarto, evaluamos con un conjunto amplio de métricas (accuracy, precision, recall, F1 y AUC-ROC), analizamos explícitamente el overfitting de cada modelo y estudiamos la importancia de las variables, interpretándola desde el contexto histórico del naufragio.

Una precisión metodológica recorre todo el análisis: el archivo **test.csv** de Kaggle no incluye la columna **Survived**, por lo que no permite calcular métricas. Por eso reservamos un 20 % de **train.csv** como conjunto de prueba (holdout), partido de forma estratificada para conservar la proporción de clases. Todas las métricas se reportan sobre ese holdout; el **test.csv** solo se usa al final para generar el archivo de submission.

2. Feature engineering aplicado

Partimos de las variables derivadas vistas en clase —**Title** (título extraído del nombre), **FamilySize** (**SibSp** + **Parch** + 1) e **IsAlone** (1 si viaja solo)— y añadimos dos variables nuevas propias, ambas implementadas dentro del pipeline para evitar fuga de información:

Fare_per_person = **Fare** / **FamilySize**. La columna **Fare** suele corresponder al precio del billete de todo el grupo que viajaba junto, no al de una sola persona. Dividirla entre el tamaño de la familia entrega el costo real por pasajero, un mejor indicador del poder adquisitivo individual. Como la riqueza se asocia a la clase y a la cercanía de los botes salvavidas, esperamos que aporte capacidad predictiva.

Deck = primera letra de **Cabin**. El número de cabina codifica la cubierta del barco (A, B, C...), que indica la posición física del pasajero y su distancia a los botes. Además, la columna **Cabin** tiene ~77 % de valores ausentes, y esa ausencia es en sí misma informativa: no tener cabina registrada se asocia a clases bajas y a menor supervivencia. Por eso, en lugar de descartar la columna, extraemos la cubierta y agrupamos los faltantes en una categoría **Unknown**, conservando la señal.

El resto del preprocesamiento también vive dentro del pipeline: las variables numéricas se imputan por mediana y se estandarizan; las categóricas se imputan por moda y se codifican con one-hot (**handle_unknown='ignore'**). Tratamos **Pclass** como categórica porque sus tres niveles no guardan una relación estrictamente lineal con la supervivencia. Como cada fit recalcula imputadores, escalador y codificador solo con datos de entrenamiento, no hay data leakage.

El EDA inicial confirma las hipótesis que guían esta ingeniería: el sexo y la clase son los predictores más marcados.

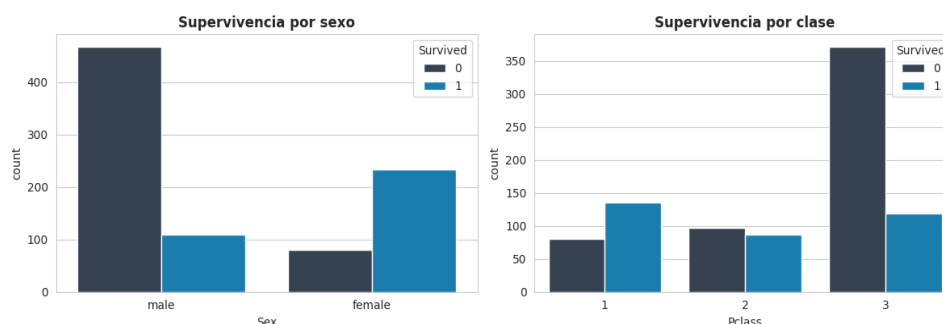


Figura 1. Supervivencia por sexo y por clase (EDA).

3. Resultados

3.1 Búsqueda de hiperparámetros

Optimizamos dos modelos con **cv=5**. Random Forest con GridSearchCV (explora todas las combinaciones de una grilla) y Gradient Boosting con RandomizedSearchCV (muestra combinaciones al azar, más eficiente). La métrica de selección fue **f1_macro**, no la accuracy: con clases desbalanceadas, la accuracy engaña —predecir siempre “no sobrevivió” ya acierta ~62 %—, mientras que el F1 macro promedia el F1 de ambas clases por igual, premiando a los modelos que también aciertan a los supervivientes, la clase minoritaria y de interés.

Modelo	Estrategia	Mejor F1 macro (CV)	Mejores hiperparámetros
Random Forest	GridSearchCV	0.807	max_depth=5, min_samples_leaf=3, n_estimators=200
Gradient Boosting	RandomizedSearchCV	0.811	n_estimators=300, max_depth=4, learning_rate=0.01

En ambos casos, la búsqueda eligió configuraciones regularizadas (profundidad limitada, hojas con más muestras, tasa de aprendizaje baja), justamente para contener el overfitting que estos modelos muestran sin restricciones.

3.2 Tabla comparativa (conjunto de prueba)

Con los mejores hiperparámetros, evaluamos los cuatro modelos sobre el holdout del 20 %:

Modelo	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	AUC-ROC
Regresión Logística	0.844	0.839	0.827	0.832	0.872
Random Forest	0.810	0.809	0.783	0.792	0.842
Gradient Boosting	0.788	0.780	0.765	0.771	0.835
Árbol de Decisión	0.765	0.758	0.736	0.743	0.827

La Regresión Logística domina en todas las métricas, con el F1 macro más alto (0.832) y el mejor AUC-ROC (0.872). Es un resultado característico de este dataset: con pocas observaciones (712 de entrenamiento) y una estructura mayormente lineal, un modelo simple y bien regularizado generaliza mejor que los ensembles. Las curvas ROC confirman el orden: las cuatro superan claramente al azar, y la curva de la logística se mantiene por encima del resto en casi todo el rango de umbrales.

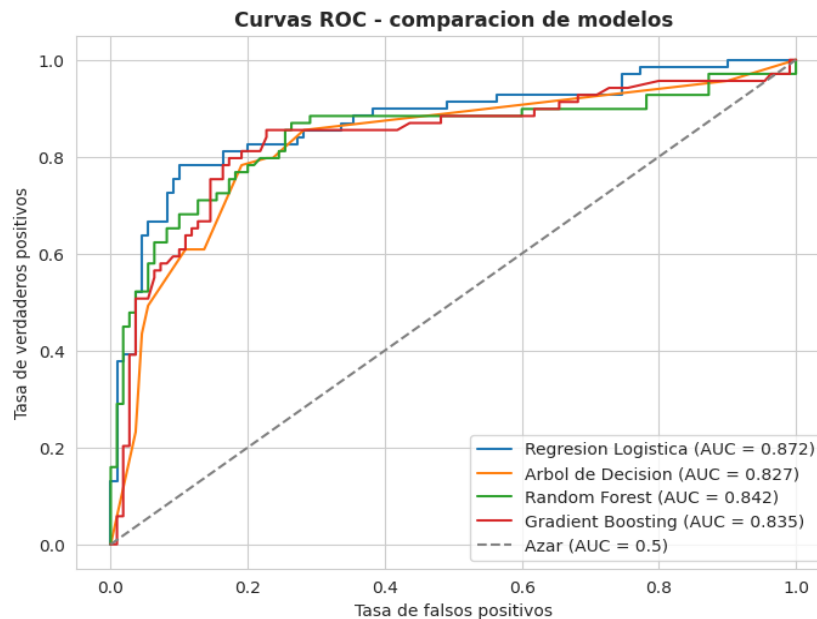


Figura 2. Curvas ROC de los cuatro modelos.

La matriz de confusión del mejor modelo muestra un comportamiento equilibrado. De 110 pasajeros que no sobrevivieron, acierta la gran mayoría (recall 0.90 en esa clase),

y de 69 supervivientes recupera el 75 %. El menor recall en la clase “Sobrevivió” es esperable por el desbalance y refleja que los errores se concentran en falsos negativos (supervivientes clasificados como fallecidos).

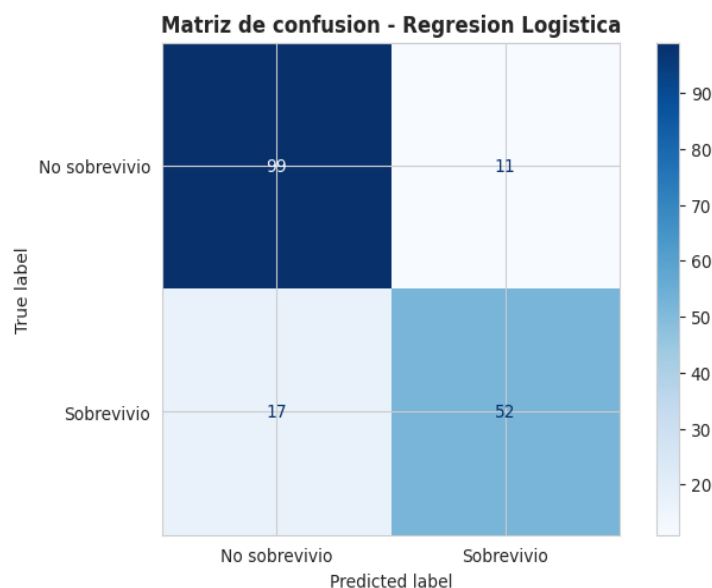


Figura 3. Matriz de confusión del mejor modelo (Regresión Logística).

3.3 Análisis de overfitting

Para cada modelo comparamos el F1 de entrenamiento con el F1 de validación cruzada (5 folds). Un gap grande (train alto, CV bajo) delata que el modelo memoriza en lugar de generalizar:

Modelo	F1 train	F1 CV (media)	Gap (train – CV)
Regresión Logística	0.827	0.812	0.015
Gradient Boosting	0.904	0.814	0.090
Random Forest	0.987	0.796	0.190
Árbol de Decisión	0.987	0.769	0.218

El Árbol de Decisión y el Random Forest muestran overfitting severo: alcanzan un F1 de entrenamiento casi perfecto (0.987) pero caen a ~0.77–0.80 en validación cruzada, con gaps de 0.218 y 0.190. Esto es típico de árboles sin límite de profundidad, que crecen hasta clasificar perfectamente cada caso de entrenamiento. Gradient Boosting

sobreajusta de forma moderada (gap 0.090), y la Regresión Logística apenas lo hace (gap 0.015): es el modelo que mejor generaliza. Cómo lo mitigamos: la búsqueda de hiperparámetros limita `max_depth` y sube `min_samples_leaf` en los árboles, y fija una `learning_rate` baja en el boosting; conviene aclarar que los valores de esta tabla corresponden a los modelos con hiperparámetros por defecto, precisamente para evidenciar el sobreajuste que motiva la optimización; las versiones ya regularizadas son las que evaluamos en la tabla comparativa de la sección 3.2. Otras vías serían reducir variables, conseguir más datos o, ante desempeños comparables, preferir el modelo con menor brecha.

3.4 Importancia de variables

Como el mejor modelo es lineal, medimos la importancia con la magnitud de los coeficientes (valor absoluto). El Top 10 lo encabezan los títulos (`Title_Mr`, `Title_Master`, `Title_Mrs`), varias cubiertas (`Deck_E`, `Deck_D`, `Deck_Unknown`), el sexo (`Sex_female`) y la clase (`Pclass_3`).

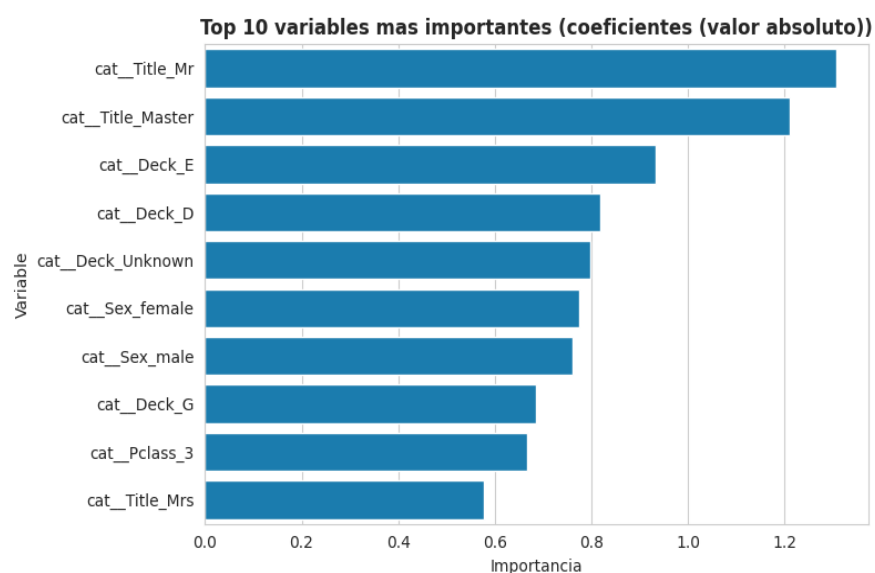


Figura 4. Top 10 de variables más importantes.

Estos resultados tienen pleno sentido desde el dominio. La política “mujeres y niños primero” explica el enorme peso de `Title_Master` (niños varones) y `Title_Mrs`, junto con `Sex_female`; en el otro extremo, `Title_Mr` (varón adulto) es el predictor más fuerte de no supervivencia. La aparición de varias cubiertas y de `Pclass_3` confirma que la posición

en el barco y la clase socioeconómica condicionaban el acceso a los botes. La variable nueva **Deck** demuestra así su aporte. El modelo reaprende de los datos la dinámica histórica documentada del naufragio.

4. Conclusiones

Modelo recomendado: Regresión Logística. Obtiene el mejor desempeño en el holdout en las cinco métricas (F1 macro 0.832, AUC-ROC 0.872) y, de forma decisiva, el menor overfitting (gap de apenas 0.015 frente a 0.090–0.218 de los demás). En un dataset pequeño y predominantemente lineal, el modelo simple generaliza mejor que Random Forest y Gradient Boosting, que memorizan el entrenamiento sin convertir esa capacidad extra en mejor desempeño fuera de muestra. La logística aporta, además, interpretabilidad directa y un costo computacional mínimo. Este hallazgo valida incluir un modelo lineal como línea base: dado que los modelos complejos no la superan, no se justifica su complejidad. Si el objetivo fuese solo maximizar el ranking, Random Forest (AUC-ROC 0.842) sería la segunda opción.

Limitaciones del análisis. Primero, el tamaño reducido del dataset (891 filas) genera alta varianza en las estimaciones, por lo que las diferencias entre modelos cercanos deben leerse con cautela. Segundo, las métricas dependen de una única partición train/test; una validación cruzada anidada daría una estimación más estable. Tercero, la variable **Cabin/Deck** tiene ~77 % de valores ausentes, así que su señal es parcial. Cuarto, no exploramos la calibración de probabilidades ni el ajuste del umbral de decisión según el costo real de cada error: un falso negativo podría considerarse más grave que un falso positivo.

Qué haríamos con más tiempo. Implementaríamos validación cruzada anidada para separar la selección de hiperparámetros de la estimación del desempeño; probaríamos stacking combinando la baja varianza de la logística con la flexibilidad de los árboles; profundizaríamos la ingeniería sobre **Ticket** y los apellidos para reconstruir los grupos de viaje; optimizaríamos el umbral de clasificación según una matriz de costos; y evaluaríamos XGBoost o LightGBM con regularización fuerte, que en datasets mayores suelen superar a los modelos aquí probados.